

Optimizer: 1D Hydraulic Simulation

Testing Harness

Filling up all required variables to test basic functionality. Can also be used to compare CFD results after extracting the required geometrical values from the CFTurbo file.

Test 1D Hydraulic Solver with CoolProp & Blade Angle Calculation

```
clear; clc; tic();
```

Fluid & Thermodynamic State (Subcooled LOX Example)

```
ParSim.fluid.name = 'Oxygen';           % CoolProp fluid string
ParSim.fluid.temperature_K = 85.0;      % Subcooled LOX Temp (K)
ParSim.fluid.pressure_Pa = 5e5;         % Suction Pressure (Pa)
```

Operating Parameters

```
ParSim.operating.Q_in = 250.0;          % Flow rate [L/min]
ParSim.operating.N_rpm = 28000;         % Shaft speed [RPM]
ParSim.operating.dp_target_bar = 75.0;   % Pressure rise target [bar]
```

Impeller & Volute Geometry (mm)

```
ParSim.pump.d2 = 78.0;                  % Impeller outlet diameter [mm]
ParSim.pump.b2 = 4.6;                   % Outlet blade width [mm]

ParSim.pump.Imp_inDia = 33.0;           % Impeller inlet diameter [mm]
ParSim.pump.b1 = 10.0;                  % Inlet blade width [mm]

ParSim.pump.Z_LA = 8;                   % Blade count [-]

ParSim.pump.d3_i = 80;                  % Volute inlet diameter [mm]
ParSim.pump.b3_i = 13;                  % Volute width [mm]

ParSim.pump.D_hub = 20.0;               % Hub diameter [mm]
ParSim.pump.deye = 33.0;               % Impeller eye diameter [mm]

ParSim.pump.alphaTH = 2.0;              % Cut-water throat angle [deg]
ParSim.pump.L_throat = 100.0;           % Throat length [mm]
```

Inducer Geometry

```
ParSim.inducer.D_tip = ParSim.pump.deye; % mm, Inducer OD matches impeller eye
ParSim.inducer.D_hub = ParSim.pump.D_hub; % mm, Inducer ID matches shaft hub
ParSim.inducer.beta_tip = 12.0;          % deg, Inducer tip blade angle
ParSim.inducer.eta_ind = 0.70;           % Initial inducer hydraulic efficiency
```

Cavitation / NPSH Parameters

```
ParSim.operating.p_suction_abs = 5e5; % Absolute suction pressure [Pa]
ParSim.operating.p_vapor = 1.2e5; % Absolute vapor pressure [Pa]
ParSim.operating.NPSH_required = 4.5; % NPSHR [m]
```

Constants & Loss Parameters

Notes:

- Many of these are extracted from the example rocket turbopump CFturbo file and serve as a good jumping off point.
- The discharge coefficient was determined by doing simulations on the example CFturbo file at different points and seeing how it varies depending on RPM; thus, the discharge coefficient is mapped to the RPM operating point value.

```
ParSim.constants.eta_h_guess = 0.50; % Initial hydraulic efficiency
guess
ParSim.constants.inlet_prewhirl = 0.0; % Pre-whirl velocity [m/s]
ParSim.constants.inlet_span = 0.5; % Mid-span calculation point
ParSim.constants.incidence_deg = 0.0; % Zero-incidence design
ParSim.constants.beta2_bounds_deg = [15 70]; % Realistic range for outlet blade
angle
ParSim.constants.y_gap = 0.50; % Tip gap clearance [mm]
ParSim.constants.yc_i = 0.50; % Volute side clearance [mm]
ParSim.constants.e1 = 1.5; % Blade thickness [mm]
ParSim.constants.cdl = 0.00012; % Leakage discharge coefficient
(NEED TO MAP THIS BASED ON RPM)
ParSim.constants.ceye = 1.00; % Eye entry coefficient
ParSim.constants.k_volute_i = 1.0e-4; % Surface roughness in volute [m]
ParSim.constants.k_impeller_i = 1.0e-4; % Surface roughness in impeller [m]
ParSim.constants.NPoints = 80; % Discretization resolution
ParSim.constants.slipLossMulti = 1.0;
```

Run calculations

1. Run 1D Axial Inducer Stage Analytical Model

```
OutInducer = InducerSolver_1D(ParSim);

% Boost suction pressure entering the centrifugal stage.
ParSim.operating.p_suction_abs = ParSim.operating.p_suction_abs +
OutInducer.dp_inducer_Pa;

% Reduce the suction pressure entering the centrifugal stage.
ParSim.operating.dp_target_bar = ParSim.operating.dp_target_bar -
OutInducer.dp_inducer_bar;
```

2. Calculate Dependent Blade Angles

```

AngleOut = calculate_pump_blade_angles(ParSim);
ParSim.pump.Beta_1 = AngleOut.Beta1_deg;
ParSim.pump.Beta_2 = AngleOut.Beta2_deg;

```

3. Run 1D Hydraulic Performance Solver

```

OutPump = HydSolver_1D(ParSim);
runtime = toc();

```

4. Display Key Outputs

```

disp('===== PUMP METRICS =====');
fprintf('Pressure Rise      : %.2f bar\n', OutPump.pressure +
OutInducer.dp_inducer_bar);
fprintf('Shaft Torque        : %.2f Nm\n', OutPump.torque);
fprintf('Hydraulic Efficiency: %.2f %\n', OutPump.efficiency * 100);
fprintf('Axial Thrust Force   : %.2f N\n', OutPump.F_axial);

disp('===== CALCULATED ANGLES =====');
fprintf('Inlet Blade Angle (Beta 1) : %.2f deg\n',
OutPump.calculated_angles.Beta1_deg);
fprintf('Outlet Blade Angle (Beta 2): %.2f deg\n',
OutPump.calculated_angles.Beta2_deg);
fprintf('Flow Deviation Angle      : %.2f deg\n', AngleOut.deviation_deg);

disp('===== INDUCER STAGE RESULTS =====');
fprintf('Inducer Axial Velocity (Cz): %.2f m/s\n', OutInducer.c_z);
fprintf('Inducer Tip Speed (Utip)   : %.2f m/s\n', OutInducer.u_tip);
fprintf('Inducer Pressure Rise     : %.2f bar\n\n', OutInducer.dp_inducer_bar);

disp('===== NASA SP-8109 SCREENING =====');
fprintf('Non-Dim Specific Speed (Ns): %.3f\n', AngleOut.Ns_nondim);
fprintf('Non-Dim Specific Dia (Ds): %.3f (Optimum: %.3f)\n', AngleOut.Ds_nondim,
AngleOut.Ds_opt);
fprintf('Ds Ratio (Ds / Ds_opt)    : %.2f\n', AngleOut.Ds_ratio);
if AngleOut.NASA_SP8109_viable
    disp('NASA SP-8109 Viability      : PASS (Geometry sits near Cordier optimum)');
else
    disp('NASA SP-8109 Viability      : WARNING (Geometry deviates significantly
from Cordier optimum)');
end

disp('===== CAVITATION CHECK =====');
fprintf('NPSH Available : %.2f m\n', OutPump.cavitation.NPSH_available_m);
fprintf('NPSH Required  : %.2f m\n', OutPump.cavitation.NPSH_required_m);
fprintf('NPSH Margin    : %.2f m\n', OutPump.cavitation.NPSH_margin_m);
fprintf('Cavitation Status: %s\n', OutPump.cavitation.status);

disp('===== RUN TIME =====');
fprintf('Execution Time : %.4f seconds\n', runtime);

```

Optimizer

In the foundational stage of this Multidisciplinary Design Optimization (MDO) pipeline, we developed a 1D mean-line analytical solver used to rapidly size a single-stage liquid oxygen (LOX) centrifugal pump and its axial inducer. Because 3D CFD simulations are computationally expensive, this 1D phase acts as a high-speed mathematical filter—ensuring only physically viable, highly efficient geometries are passed downstream to CFTurbo and ANSYS.

Methodology and Execution

We built a custom 1D script relying on Euler's turbomachinery equations and fundamental velocity triangles to predict hydraulic efficiency, pressure rise, and shaft torque. Real-fluid thermodynamics for subcooled LOX (via CoolProp fallback standards) were integrated to account for density shifts and accurately map vapor pressure.

Rather than relying on guess-and-check methodologies, we wrapped this 1D solver inside a parallelized Genetic Algorithm (GA). The optimizer was programmed to minimize a proportional penalty function, evaluating hundreds of thousands of candidate geometries against three strict physical and empirical constraints.

Core Governing Constraints

Empirical Geometry Limits (The NASA SP-8109 Cordier Line)

To prevent the optimizer from generating nonphysical impeller shapes (e.g., excessively wide or narrow profiles), designs were bench marked against historical aerospace pumps using two non-dimensional parameters:

- **Specific Speed (N_s):** Defines the optimal flow path shape.

$$N_s = \frac{\omega \cdot \sqrt{Q}}{(g \cdot H)^{3/4}}$$

- **Specific Diameter (D_s):** Evaluates the physical size of the impeller.

$$D_s = \frac{d_2 \cdot (g \cdot H)^{1/4}}{\sqrt{Q}}$$

The solver calculated the optimum Specific Diameter ($D_{s,opt}$) using the empirical Cordier curve. Geometries were strictly penalized if they fell outside the NASA SP-8109 tolerance band, forcing the algorithm to discard evolutionary branches that drifted off the optimum ridge:

$$0.75 \leq \frac{D_{s,actual}}{D_{s,opt}} \leq 1.35$$

Cavitation Mitigation (NPSH Limits)

To ensure the subcooled LOX does not boil as it accelerates into the pump, a hard cavitation constraint was applied to the inducer and main impeller eye. This is called the Net Positive Suction Head Available ($NPSH_A$):

$$NPSH_A = \frac{P_{suction} - P_{vapor}}{\rho \cdot g} + \frac{C_m^2}{2g}$$

The required NPSH ($NPSH_R$) was calculated based on the inducer's empirical suction specific speed. Any design failing the fundamental cavitation check triggered a massive flat penalty (+200), instantly killing the geometry's viability:

$$NPSH_A \geq NPSH_R + \text{Safety Margin}$$

Target Performance & The Fitness Landscape

To ensure the pump meets the exact requirements of the rocket engine, the solver calculated the actual pressure rise generated by the velocity triangles:

$$\Delta P_{actual} = \rho \cdot g \cdot H_{euler} \cdot \eta_h$$

The objective function applied a proportional penalty based on how far the actual pressure drifted from the pressure target. This provided a continuous "gradient" for the Genetic Algorithm to follow down to the optimum design point:

$$\text{Penalty} = 15 \times |\Delta P_{actual} - \Delta P_{target}|$$

Optimizer using Global Optimization Toolbox

Initialization

```
clear; clc; close all;
```

Initialize Parallel Pool

```
if isempty(gcp('nocreate'))
    fprintf('Starting Parallel Pool...\n');
    parpool;
else
    fprintf('Parallel Pool already running.\n');
end
```

Define Decision Variables & Tightened Bounds (RATIO BASED)

Variable Vector: $x = [d2, b2_ratio, deye_ratio, b1_ratio, Z, N_rpm]$

Note: We tighten the bounds around your working 78mm seed to give the GA a physically viable "sandbox" to explore.

```
lb = [70.0, 0.040, 0.45, 0.20, 5, 28000];
ub = [90.0, 0.080, 0.50, 0.40, 9, 40000];

IntCon = 5; % Z (Blade Count) must be an integer
```

Optimization Setup

```
TargetParams.Q_in = 250.0;
```

```

TargetParams.dp_target_bar = 75.0;
TargetParams.NPSH_required = 4.5;
TargetParams.fluid = 'Oxygen';

% --- THE SEED ---
x_seed = [78.0, 0.059, 0.423, 0.303, 8, 35000];

% Configure Genetic Algorithm
opts = optimoptions('ga', ...
    'InitialPopulationMatrix', x_seed, ...
    'UseParallel', true, ...
    'UseVectorized', false, ...
    'PopulationSize', 1000, ...
    'MaxGenerations', 300, ...
    'MaxStallGenerations', 100, ...
    'FunctionTolerance', 1e-6, ...
    'Display', 'diagnose', ...
    'PlotFcn', {@gaplotbestf, @gaplotstopping});

```

Run Optimization

```

fprintf('\nStarting Parallel Optimization (300 Generations)... \n');
tic;

% Pass the TargetParams into the objective function wrapper
FitnessFcn = @(x) turbopump_objective(x, TargetParams);
[x_opt, fval_opt, exitflag, output] = ga(FitnessFcn, 6, [], [], [], [], lb, ub, [],
IntCon, opts);

ExecutionTime = toc;
fprintf('\nOptimization Completed in %.2f seconds.\n', ExecutionTime);

```

Display Optimal Results

Note: Convert Ratios back to Physical Millimeters for Printing

```

d2_opt = x_opt(1);
b2_opt = max(d2_opt * x_opt(2), 2.0);
deye_opt = max(d2_opt * x_opt(3), 22.0);
b1_opt = max(deye_opt * x_opt(4), 2.0);
Z_opt = round(x_opt(5));
N_rpm_opt = x_opt(6);

```

Local Objective Function

```

function score = turbopump_objective(x, Target)

```

Decode Candidate Variables from Ratios

```

d2 = x(1);
b2 = max(d2 * x(2), 2.0);

```

```

deye = max(d2 * x(3), 22.0);
b1    = max(deye * x(4), 2.0);
Z     = round(x(5));
N     = x(6);

```

Build the ParSim Structure for this iteration

```

ParSim.fluid.name = 'custom'; % Force custom to bypass CoolProp in parallel pool
ParSim.fluid.temperature_K = 90.0;
ParSim.fluid.pressure_Pa = 5e5;

ParSim.operating.Q_in = Target.Q_in;
ParSim.operating.N_rpm = N;
ParSim.operating.dp_target_bar = Target.dp_target_bar;
ParSim.operating.p_suction_abs = 5e5;
ParSim.operating.p_vapor = 1.2e5;
ParSim.operating.NPSH_required = Target.NPSH_required;

% Geometry mapping
ParSim.pump.d2 = d2;
ParSim.pump.b2 = b2;
ParSim.pump.deye = deye;
ParSim.pump.Imp_inDia = deye; % Assuming eye matches inlet
ParSim.pump.b1 = b1;
ParSim.pump.Z_LA = Z;

% Fixed Geometry (Not optimized by GA to save time)
ParSim.pump.d3_i = d2 + 2.0; % Volute slightly larger than d2
ParSim.pump.b3_i = b2 * 2.5; % Standard volute width rule of thumb
ParSim.pump.D_hub = 20.0;
ParSim.pump.alphaTH = 2.0;
ParSim.pump.L_throat = 100.0;

ParSim.inducer.D_tip = deye;
ParSim.inducer.D_hub = 20.0;
ParSim.inducer.beta_tip = 12.0;
ParSim.inducer.eta_ind = 0.70;

```

Run the Solvers inside a Try-Catch

```

try
    % Inducer
    OutInducer = InducerSolver_1D(ParSim);

    % EXPLOIT FIX: If inducer acts as a turbine (negative pressure), kill it
    instantly
    if OutInducer.dp_inducer_bar <= 0
        error('Inducer turbine effect detected.');
```

```

    ParSim.operating.p_suction_abs = ParSim.operating.p_suction_abs +
    OutInducer.dp_inducer_Pa;
    ParSim.operating.dp_target_bar = ParSim.operating.dp_target_bar -
    OutInducer.dp_inducer_bar;

    % Angles
    AngleOut = calculate_pump_blade_angles(ParSim);
    ParSim.pump.Beta_1 = AngleOut.Beta1_deg;
    ParSim.pump.Beta_2 = AngleOut.Beta2_deg;

    % Main Solver
    OutPump = HydSolver_1D(ParSim);

    % Extract Results
    eff = OutPump.eta;
    dP_actual = OutPump.pressure + OutInducer.dp_inducer_bar;
    Ds_ratio = AngleOut.Ds_ratio;
    cav_risk = OutPump.cavitation.cavitation_risk;

    % If angles fail, solver outputs NaNs or <10% eff. We must catch it here!
    if isnan(eff) || isnan(dP_actual) || eff < 0.10
        error('Solver returned fallback garbage due to impossible velocity
triangles.');
```

```

    end
catch
```

```

    % If geometry fails physics checks, assign terrible baseline stats
    eff = -1.0;
    dP_actual = 1.0;
    Ds_ratio = 10.0;
    cav_risk = true;
end
```

Calculate Proportional Fitness Score

Note: Base Score is to Maximize efficiency (Minimize negative efficiency)

```

base_score = abs(1.0 - eff) * 100; % E.g., 65% efficiency -> -65 score

% Proportional Pressure Penalty
dP_penalty = 15 * abs(dP_actual - Target.dp_target_bar);

% Proportional Ds Ratio Penalty (Keep it within Cordier limits)
Ds_penalty = 0;
if Ds_ratio < 0.75
    Ds_penalty = 100 * abs(0.75 - Ds_ratio);
elseif Ds_ratio > 1.35
    Ds_penalty = 100 * abs(Ds_ratio - 1.35);
end

% Cavitation Penalty
cav_penalty = 0;
```

```

if cav_risk
    cav_penalty = 200;
end

% Final Score (Lower is better)
score = base_score + dP_penalty + Ds_penalty + cav_penalty;
end

```

Display Optimal Results

Note: Convert Ratios back to Physical Millimeters for Printing

```

d2_opt = x_opt(1);
b2_opt = max(d2_opt * x_opt(2), 2.0);
deye_opt = max(d2_opt * x_opt(3), 22.0);
b1_opt = max(deye_opt * x_opt(4), 2.0);
Z_opt = round(x_opt(5));
N_rpm_opt = x_opt(6);

fprintf('\n=====');
fprintf('          OPTIMAL PUMP GEOMETRY\n');
fprintf('=====');
fprintf('Impeller OD (d2)      : %.2f mm\n', d2_opt);
fprintf('Outlet Width (b2)     : %.2f mm\n', b2_opt);
fprintf('Eye Diameter (deye)   : %.2f mm\n', deye_opt);
fprintf('Inlet Width (b1)      : %.2f mm\n', b1_opt);
fprintf('Blade Count (Z)       : %d\n', Z_opt);
fprintf('Shaft Speed (N)       : %.0f RPM\n', N_rpm_opt);
fprintf('Final Penalty Score : %.2f\n', fval_opt);
fprintf('=====');

```

Re-Run Full Simulation on Optimal Point for Detailed Stats

Build ParSim for the optimum point

```

ParSim.fluid.name = 'Oxygen';           % Re-enable CoolProp for the final accurate
check
ParSim.fluid.temperature_K = 90.0;     % Match target parameters
ParSim.fluid.pressure_Pa = 5e5;

ParSim.operating.Q_in = TargetParams.Q_in;
ParSim.operating.N_rpm = N_rpm_opt;
ParSim.operating.dp_target_bar = TargetParams.dp_target_bar;
ParSim.operating.p_suction_abs = 5e5;
ParSim.operating.p_vapor = 1.2e5;
ParSim.operating.NPSH_required = TargetParams.NPSH_required;

ParSim.pump.d2 = d2_opt;
ParSim.pump.b2 = b2_opt;
ParSim.pump.deye = deye_opt;
ParSim.pump.Imp_inDia = deye_opt;

```

```

ParSim.pump.b1 = b1_opt;
ParSim.pump.Z_LA = Z_opt;

% Fixed dimensions
ParSim.pump.d3_i = d2_opt + 2.0;
ParSim.pump.b3_i = b2_opt * 2.5;
ParSim.pump.D_hub = 20.0;
ParSim.pump.alphaTH = 2.0;
ParSim.pump.L_throat = 100.0;

ParSim.inducer.D_tip = deye_opt;
ParSim.inducer.D_hub = 20.0;
ParSim.inducer.beta_tip = 12.0;
ParSim.inducer.eta_ind = 0.70;

```

Run the 3 solvers sequentially

```

OutInducer = InducerSolver_1D(ParSim);
ParSim.operating.p_suction_abs = ParSim.operating.p_suction_abs +
OutInducer.dp_inducer_Pa;
ParSim.operating.dp_target_bar = ParSim.operating.dp_target_bar -
OutInducer.dp_inducer_bar;

AngleOut = calculate_pump_blade_angles(ParSim);
ParSim.pump.Beta_1 = AngleOut.Beta1_deg;
ParSim.pump.Beta_2 = AngleOut.Beta2_deg;

OutPump = HydSolver_1D(ParSim);

```

Print Final Stats (Using requested format)

```

disp('===== PUMP METRICS =====');
fprintf('Pressure Rise           : %.2f bar\n', OutPump.pressure +
OutInducer.dp_inducer_bar);
fprintf('Shaft Torque              : %.2f Nm\n', OutPump.torque);
fprintf('Hydraulic Efficiency: %.2f %%\n', OutPump.efficiency * 100);
fprintf('Axial Thrust Force   : %.2f N\n', OutPump.F_axial);

disp('===== CALCULATED ANGLES =====');
fprintf('Inlet Blade Angle (Beta 1) : %.2f deg\n',
OutPump.calculated_angles.Beta1_deg);
fprintf('Outlet Blade Angle (Beta 2): %.2f deg\n',
OutPump.calculated_angles.Beta2_deg);
fprintf('Flow Deviation Angle      : %.2f deg\n', AngleOut.deviation_deg);

disp('===== INDUCER STAGE RESULTS =====');
fprintf('Inducer Axial Velocity (Cz): %.2f m/s\n', OutInducer.c_z);
fprintf('Inducer Tip Speed (Utip)   : %.2f m/s\n', OutInducer.u_tip);
fprintf('Inducer Pressure Rise     : %.2f bar\n\n', OutInducer.dp_inducer_bar);

```

```

disp('===== NASA SP-8109 SCREENING =====');
fprintf('Non-Dim Specific Speed (Ns): %.3f\n', AngleOut.Ns_nondim);
fprintf('Non-Dim Specific Dia (Ds): %.3f (Optimum: %.3f)\n', AngleOut.Ds_nondim,
AngleOut.Ds_opt);
fprintf('Ds Ratio (Ds / Ds_opt) : %.2f\n', AngleOut.Ds_ratio);
if AngleOut.NASA_SP8109_viable
    disp('NASA SP-8109 Viability : PASS (Geometry sits near Cordier optimum)');
else
    disp('NASA SP-8109 Viability : WARNING (Geometry deviates significantly
from Cordier optimum)');
end

disp('===== CAVITATION CHECK =====');
fprintf('NPSH Available : %.2f m\n', OutPump.cavitation.NPSH_available_m);
fprintf('NPSH Required : %.2f m\n', OutPump.cavitation.NPSH_required_m);
fprintf('NPSH Margin : %.2f m\n', OutPump.cavitation.NPSH_margin_m);
fprintf('Cavitation Status: %s\n', OutPump.cavitation.status);

```

1D Axial Inducer Simulation

The axial inducer that leads into the impeller eye is responsible for 8-12% of the energy addition in the turbopump system. It also, crucially, helps avoid cavitation by increasing the pressure into the impeller which also contributes to a more efficient pressure rise. This script is a mean-line analytical model for the inducer based on the Euler pump equation and is intended to be run before the main centrifugal pump calculations.

Key Governing Equations

For an axial inducer, the theoretical Euler Head (H_{ind}) is calculated using the blade tip speed (U_{tip}), the axial flow velocity (C_z), and the inducer tip blade angle (β_{tip}):

Axial Velocity

$$C_z = \frac{Q}{A_{eye}}$$

Tip Tangential Velocity

$$C_{\theta 2} = U_{tip} - \frac{C_z}{\tan(\beta_{tip})}$$

Inducer Head

$$H_{ind} = \frac{U_{tip} \cdot C_{\theta 2}}{g}$$

Underlying Assumptions

Conservative efficiency estimate

When we convert the calculated head into a an actual pressure rise (ΔP_{ind}) using a conservative inducer hydraulic efficiency estimate ($\eta_{ind} \approx 70$, as inducers are designed for cavitation resistance not peak efficiency).

Run 1D Axial Inducer Solver

```
function OutInducer = InducerSolver_1D(ParSim)
```

Kinematics and Flow Area

```
A_ind_m2 = (pi/4) * ((ParSim.inducer.D_tip/1000)^2 - (ParSim.inducer.D_hub/1000)^2);
Q_m3s = ParSim.operating.Q_in * 1e-3 / 60;
c_z = Q_m3s /
A_ind_m2;                                     % Axial flow
velocity [m/s]
u_tip = (pi * ParSim.operating.N_rpm * (ParSim.inducer.D_tip/1000)) /
60;      % Tip speed [m/s]

OutInducer.c_z = c_z;
OutInducer.u_tip = u_tip;
```

Velocity Triangle and Euler Head

Note: Assuming zero pre-whirl entering the inducer (C_{u1}).

```
c_u2_ind = u_tip - (c_z / tan(deg2rad(ParSim.inducer.beta_tip)));
H_inducer = (u_tip * c_u2_ind) / 9.81;      % Ideal Euler Head [m]
```

Calculating Inducer Pressure Rise

```
fluidProps = getFluidProperties(ParSim);
rho_propellant = fluidProps.rho;
dp_inducer_Pa = rho_propellant * 9.81 * H_inducer * ParSim.inducer.eta_ind;
dp_inducer_bar = dp_inducer_Pa / 1e5;

OutInducer.dp_inducer_Pa = dp_inducer_Pa;
OutInducer.dp_inducer_bar = dp_inducer_bar;

end
```

Calculating Pump's Blade Angles

This script is an inverse-design pre-processor routine. It calculates the geometric blade metal angles at the impeller inlet (β_{1B}) and outlet (β_{2B}) prior to running full hydraulic performance sweeps. This step ensures that the generated impeller geometry satisfies target head requirements under real fluid slip and prescribed flow incidence conditions.

Key Governing Equations

Inlet Velocity Triangle & Relative Blade Angle

Meridional velocity at the inlet eye is calculated by accounting for hub/tip dimensions and blade thickness blockage:

$$A_1 = \frac{\pi}{4} (D_{\text{eye}}^2 - D_{\text{hub}}^2) \cdot \lambda_1$$

$$c_{m1} = \frac{Q}{A_1}$$

$$\beta_{1,\text{flow}} = \arctan\left(\frac{c_{m1}}{u_1 - c_{u1}}\right)$$

$$\beta_{1B} = \beta_{1,\text{flow}} + i$$

where $\lambda_1 = \max\left(1 - \frac{Zt}{\pi D_{\text{mean}}}, 0.05\right)$, $u_1 = \omega r_{1,\text{local}}$, c_{u1} is the inlet pre-whirl velocity, and i is the specified incidence angle.

These equations are derived from the velocity triangle which are as follows:

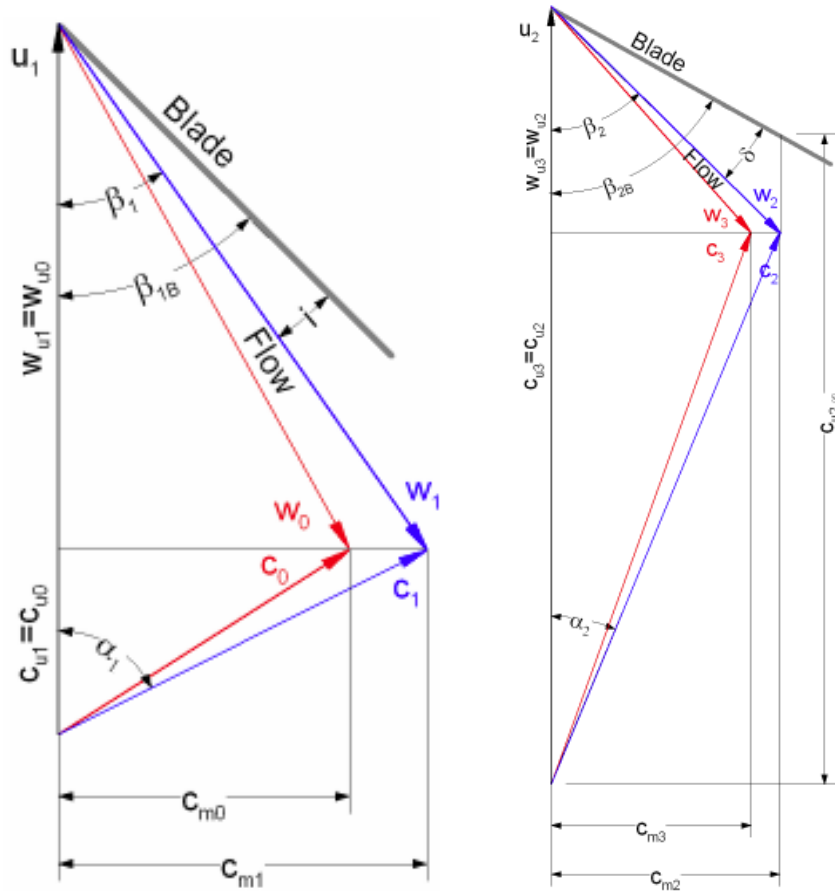


Figure 1a and 1b. Inlet velocity and outlet velocity triangle (CFturbo manual).

Required Euler Head & Target Tangential Velocity

$$H_{e,\text{req}} = \frac{H_{\text{target}}}{\eta_{h,\text{guess}}}$$

$$c_{u2,\text{req}} = \frac{gH_{e,\text{req}} + u_1 c_{u1}}{u_2}$$

Slip Factor Coupling & Outlet Angle Residual Solver

The required outlet blade metal angle β_{2B} is determined implicitly using a 1D root-finding algorithm (fzero) that balances the required tangential velocity against slip-corrected ideal whirl:

$$R(\beta_{2B}) = \sigma(\beta_{2B}, Z, D_1/D_2, \beta_{1B}) \cdot c_{u2,\text{ideal}}(\beta_{2B}) - c_{u2,\text{req}} = 0$$

$$c_{u2,\text{ideal}}(\beta_{2B}) = u_2 - c_{m2} \cot(\beta_{2B})$$

EI-Naggar Base Slip Factor Correlation (σ):

$$\sigma_0 = 1 - \frac{\sqrt{\sin(\pi - \beta_{2B})}}{Z^{0.7}} \left[1 - \left(\frac{d_1/d_2 - e_{\text{limit}}}{1 - e_{\text{limit}}} \right)^{1/3} \frac{1}{\pi - \beta_{1B}} \right]$$

where $e_{\text{limit}} = \exp\left(\frac{-8.16 \sin(\pi - \beta_{2B})}{Z}\right)$.

Underlying Assumptions

Uniform Axisymmetric Flow

Velocities (c_{m1}, c_{m2}) are assumed uniform across the inlet span and discharge width.

Constant Initial Efficiency Guess

Required Euler head relies on a constant initial hydraulic efficiency estimate ($\eta_{h,\text{guess}}$). The guess is extracted from the example rocket turbopump CFturbo file.

1D Slip Representation

Blade-to-blade flow deviation is captured entirely by empirical slip factor correlations without modeling 3D secondary flow structures.

Linear Blockage Approximation

Metal blockage at the inlet and outlet is approximated using standard circumferential blade thickness ratios.

Run Blade Angle Calculations

```
function AngleOut = calculate_pump_blade_angles(ParSim)

validateRequiredInputs(ParSim);

g = 9.81;
N = ParSim.operating.N_rpm;
```

Get dynamic thermodynamic fluid properties

```
fluidProps = getFluidProperties(ParSim);
rho = fluidProps.rho;
Q = ParSim.operating.Q_in * 1e-3 / 60; % L/min to m^3/s
```

Geometric and operating properties

```
eta_h_guess = getConst(ParSim, 'eta_h_guess', 0.64);
cu1 = getConst(ParSim, 'inlet_prewirl', 0.0);
span = getConst(ParSim, 'inlet_span', 0.5);
incidence_deg = getConst(ParSim, 'incidence_deg', 0.0);
beta2_bounds_deg = getConst(ParSim, 'beta2_bounds_deg', [15 70]);

Deye = ParSim.pump.deye * 1e-3;
Dhub = ParSim.pump.D_hub * 1e-3;
D2 = ParSim.pump.d2 * 1e-3;
b2 = ParSim.pump.b2 * 1e-3;
Z = ParSim.pump.Z_LA;
t = getConst(ParSim, 'e1', 1.5) * 1e-3;

if Deye <= Dhub
    error('calculate_pump_blade_angles:BadEyeGeometry', 'deye must be greater than D_hub.');
```

end

```
if span < 0 || span > 1
    error('calculate_pump_blade_angles:BadSpan', 'inlet_span must be between 0 and 1.');
```

end

```
if eta_h_guess <= 0 || eta_h_guess > 1
    error('calculate_pump_blade_angles:BadEfficiency', 'eta_h_guess must be between 0 and 1.');
```

end

```
omega = 2 * pi * N / 60;
```

Inlet Blade Angle (Zero-Incidence or Specified Incidence)

```
if hasConst(ParSim, 'inlet_blockage')
    blockage1 = getConst(ParSim, 'inlet_blockage', 1.0);
else
    DmeanEye = 0.5 * (Deye + Dhub);
    blockage1 = max(1 - (Z * t / (pi * DmeanEye)), 0.05);
end

A1 = (pi / 4) * (Deye^2 - Dhub^2) * blockage1;
cm1 = Q / A1;
D1_local = Dhub + span * (Deye - Dhub);
r1_local = D1_local / 2;
u1 = omega * r1_local;

beta1_flow = atan2(cm1, max(u1 - cu1, eps));
beta1B = beta1_flow + deg2rad(incidence_deg);
```

Outlet Blade Angle from Required Euler Head and Slip Correlation

```
if hasConst(ParSim, 'outlet_blockage')
    blockage2 = getConst(ParSim, 'outlet_blockage', 1.0);
else
    blockage2 = max(1 - (Z * t / (pi * D2)), 0.05);
end

A2 = pi * D2 * b2 * blockage2;
cm2 = Q / A2;
u2 = omega * D2 / 2;

H_target = getTargetHead(ParSim, rho, g);
H_euler_required = H_target / eta_h_guess;
cu2_required = (g * H_euler_required + u1 * cu1) / u2;

beta_lo = deg2rad(beta2_bounds_deg(1));
beta_hi = deg2rad(beta2_bounds_deg(2)) + pi;

residual = @(b2B) outletWhirlResidual(b2B, u2, cm2, cu2_required, Z, Dhub/D2,
beta1B);
f_lo = residual(beta_lo);
f_hi = residual(beta_hi);

if ~isfinite(f_lo) || ~isfinite(f_hi) || sign(f_lo) == sign(f_hi)
    beta_grid = linspace(beta_lo, beta_hi, 300);
    f_grid = arrayfun(residual, beta_grid);
    idx = find(isfinite(f_grid(1:end-1)) & isfinite(f_grid(2:end)) &
(sign(f_grid(1:end-1)) .* sign(f_grid(2:end)) <= 0), 1, 'first');
    if isempty(idx)
        error('calculate_pump_blade_angles:NoBeta2Solution', ...
            'No beta2 solution exists inside %.1f to %.1f deg. Check target head,
eta_h_guess, D2, b2, and RPM.', ...
            beta2_bounds_deg(1), beta2_bounds_deg(2));
    end
    beta_lo = beta_grid(idx);
    beta_hi = beta_grid(idx+1);
end

beta2B = pi - fzero(residual, [beta_lo, beta_hi]);
sigma = originalBaseSlipFactor(beta2B, Z, Dhub/D2, beta1B);
cu2_ideal = u2 - cm2 * cot(beta2B);
cu2_actual = sigma * cu2_ideal;
beta2_flow = atan2(cm2, max(u2 - cu2_actual, eps));
deviation = beta2B - beta2_flow;
```

Notes on NASA SP-8109

NASA SP-8109 (*Liquid Rocket Engine Centrifugal Flow Turbopumps*, published in 1973) is a comprehensive design monograph compiled during the Apollo era. It synthesizes decades of empirical flight and test-stand

data from historical liquid engine turbopumps (such as the F-1, J-2, RL10, and Titan engines) into standardized design guidelines, loss trends, and geometric scaling laws. Its core concept is the Cordier / Balje ($N_s - D_s$) Optimal Line. When designing a turbopump, your mission requirements usually fix three variables: **required flow rate (Q)**, **required pressure rise/head (H)**, and **shaft rotational speed (N)**.

To determine the ideal physical dimensions of the impeller without blind guessing, NASA SP-8109 uses two non-dimensional numbers:

1. Non-Dimensional Specific Speed (N_s): Describes the hydraulic "type" of the pump (radial, mixed-flow, or axial):

$$N_s = \frac{\omega \sqrt{Q}}{(gH)^{0.75}}$$

2. Non-Dimensional Specific Diameter (D_s): Relates the outer impeller diameter (D_2) to flow and head:

$$D_s = \frac{D_2(gH)^{0.25}}{\sqrt{Q}}$$

When thousands of tested pumps are plotted on an N_s vs. D_s chart, peak hydraulic efficiency collapses along a narrow curve known as the **Cordier Line**:

$$D_s \approx \frac{2.16}{N_s^{0.6}} \cdot 2$$

By integrating this, the code is able to flag candidates that deviate significantly from optimal proportions ($D_s / D_{s,opt}$ ratio outside 0.75 – 1.35) before running full loss iterations. It also, crucially, allows the sizing of an initial diameter (D_2) given an RPM, or vice versa.

NASA SP-8109 Specific Speed & Diameter Check

```

Ns_nondim = omega * sqrt(Q) / (g * H_target)^0.75; % Non-dimensional Ns
Ds_nondim = D2 * (g * H_target)^0.25 / sqrt(Q);    % Non-dimensional Ds
Ds_opt = (2.16 / (Ns_nondim^0.6)) * 2;             % NASA SP-8109 / Cordier
optimum fit
Ds_ratio = Ds_nondim / Ds_opt;                     % Near 1.0 is optimal

```

Package Results

```

AngleOut.Beta1_deg = rad2deg(beta1B);
AngleOut.Beta2_deg = rad2deg(beta2B);
AngleOut.Beta1_flow_deg = rad2deg(beta1_flow);
AngleOut.Beta2_flow_deg = rad2deg(beta2_flow);
AngleOut.incidence_deg = rad2deg(beta1B - beta1_flow);
AngleOut.deviation_deg = rad2deg(deviation);
AngleOut.sigma_design = sigma;
AngleOut.H_target_m = H_target;
AngleOut.H_euler_required_m = H_euler_required;
AngleOut.cu2_required_mps = cu2_required;

```

```

AngleOut.cu2_actual_mps = cu2_actual;
AngleOut.cm1_mps = cm1;
AngleOut.cm2_mps = cm2;
AngleOut.u1_mps = u1;
AngleOut.u2_mps = u2;
AngleOut.A1_m2 = A1;
AngleOut.A2_m2 = A2;
AngleOut.blockage1 = blockage1;
AngleOut.blockage2 = blockage2;
AngleOut.inlet_span = span;
AngleOut.deviation_warning = abs(AngleOut.deviation_deg) > 14;

```

NASA SP-8109 screening flags

```

AngleOut.Ns_nondim = Ns_nondim;
AngleOut.Ds_nondim = Ds_nondim;
AngleOut.Ds_opt = Ds_opt;
AngleOut.Ds_ratio = Ds_ratio;
AngleOut.NASA_SP8109_viable = (Ds_ratio >= 0.75 && Ds_ratio <= 1.35);

```

end

Helper Functions

```

function r = outletWhirlResidual(beta2B, u2, cm2, cu2Required, Z, d1d2, beta1B)
    sigma = originalBaseSlipFactor(beta2B, Z, d1d2, beta1B);
    cu2Ideal = u2 - cm2 * cot(beta2B);
    r = sigma * cu2Ideal - cu2Required;

```

end

```

function sigma0 = originalBaseSlipFactor(beta2B, Z, d1d2, beta1B)
    beta2Corr = pi - beta2B;
    beta1Corr = pi - beta1B;
    eLimit = exp(-8.16 * sin(beta2Corr) / Z);
    if eLimit >= d1d2
        sigma0 = 1 - sqrt(sin(beta2Corr)) / (Z^0.7);
    else
        ratioTerm = max((d1d2 - eLimit) / (1 - eLimit), 0);
        sigma0 = 1 - (sqrt(sin(beta2Corr)) * (1 - ratioTerm^(1/3))) / (beta1Corr *
(Z^0.7));

```

end

sigma0 = min(max(sigma0, 0.01), 0.999);

end

```

function H = getTargetHead(ParSim, rho, g)
    if isfield(ParSim.operating, 'H_target_m')
        H = ParSim.operating.H_target_m;
    elseif isfield(ParSim.operating, 'dp_target_Pa')
        H = ParSim.operating.dp_target_Pa / (rho * g);
    elseif isfield(ParSim.operating, 'dp_target_bar')

```

```

        H = ParSim.operating.dp_target_bar * 1e5 / (rho * g);
    elseif isfield(ParSim.operating, 'P_in')
        H = ParSim.operating.P_in * 1e5 / (rho * g);
    else
        error('calculate_pump_blade_angles:MissingTarget', 'Provide H_target_m,
dp_target_Pa, or dp_target_bar.');
```

Notes on CoolProp

CoolProp is a high-accuracy thermophysical property database (similar to NIST REFPROP). It uses High-Accuracy Helmholtz Energy Equations of State (HEOS) to evaluate real-fluid properties across subcooled, saturated, two-phase, and super-critical regimes. In liquid rocket engines, propellants like **Liquid Oxygen (LOX)**, **Liquid Methane (LCH₄)**, and **Liquid Hydrogen (LH₂)** behave very differently:

- **Subcooling & Density Sensitivity:** Subcooled LOX density changes noticeably with temperature. A 5 K shift in propellant feed temperature alters fluid density (ρ), directly changing the volumetric flow rate ($Q = \dot{m}/\rho$) for a fixed mass flow. This requires a different inlet eye radius (r_{in}) and tip radius (R_{out}) to maintain velocity triangles.
- **Vapor Pressure & Cavitation (NPSH):** Vapor pressure (P_{vap}) rises exponentially with temperature. If local pressure inside the blade passage drops below P_{vap} , vapor bubbles form and collapse violently (cavitation), destroying performance and damaging hardware.

$$NPSH_{available} = \frac{P_{suction,abs} - P_{vapor}(T)}{\rho(T, P) \cdot g} + \frac{v_{suction}^2}{2g}$$

By integrating CoolProp into the code, the following benefits are realized:

- **Dynamic P_{vap} :** The code evaluates P_{vap} dynamically based on fluid temperature T , producing an accurate Net Positive Suction Head (NPSHa) assessment.
- **Mass Flow $\longleftrightarrow$ Volume Flow Link:** The solver evaluates fluid state (T, P) to convert mass flow rates into true volumetric velocity vectors (c_{m1}, c_{m2}).
- **Multi-Propellant Flexibility:** You can switch the fluid input string in `ParSim.fluid.name` between 'Oxygen', 'Methane', 'Hydrogen', or 'RP-1' without needing to manually look up and rewrite fluid constants. Though, this would be outside the scope of this particular project.

```

function props = getFluidProperties(ParSim)
    % CoolProp dynamic thermal database integration with static fallback
    if isfield(ParSim, 'fluid') && isfield(ParSim.fluid, 'name') &&
~strcmp(ParSim.fluid.name, 'custom')
        try
            fluid = ParSim.fluid.name;
            T_K = ParSim.fluid.temperature_K;
            P_Pa = ParSim.fluid.pressure_Pa;
```

```

        % Call CoolProp via MATLAB Python interface
        props.rho = double(py.CoolProp.CoolProp.PropsSI('D', 'T', T_K, 'P',
P_Pa, fluid));
        mu = double(py.CoolProp.CoolProp.PropsSI('V', 'T', T_K, 'P', P_Pa,
fluid));
        props.nu = mu / props.rho;
        props.p_vapor = double(py.CoolProp.CoolProp.PropsSI('P', 'T', T_K, 'Q',
0, fluid));
        return;
    catch
        warning('CoolProp call failed or unconfigured. Falling back to constant
inputs.');
```

```

    end
end

% Fallback to user-defined constants
props.rho = getConst(ParSim, 'rho_propellant', 1049.0);
props.nu = getConst(ParSim, 'Nu', 5.0e-7);
props.p_vapor = getConst(ParSim, 'p_vapor_default', 1.0e5);
end

function value = getConst(ParSim, name, defaultValue)
    if isfield(ParSim, 'constants') && isfield(ParSim.constants, name)
        value = ParSim.constants.(name);
    else
        value = defaultValue;
    end
end

function tf = hasConst(ParSim, name)
    tf = isfield(ParSim, 'constants') && isfield(ParSim.constants, name);
end

function validateRequiredInputs(ParSim)
    requiredOperating = {'N_rpm', 'Q_in'};
    requiredPump = {'deye', 'D_hub', 'd2', 'b2', 'Z_LA'};
    for k = 1:numel(requiredOperating)
        if ~isfield(ParSim, 'operating') || ~isfield(ParSim.operating,
requiredOperating{k})
            error('calculate_pump_blade_angles:MissingInput', 'Missing
ParSim.operating.%s', requiredOperating{k});
        end
    end
    for k = 1:numel(requiredPump)
        if ~isfield(ParSim, 'pump') || ~isfield(ParSim.pump, requiredPump{k})
            error('calculate_pump_blade_angles:MissingInput', 'Missing
ParSim.pump.%s.', requiredPump{k});
        end
    end
end
end

```

1D Hydraulic Pump Simulation

This script is a forward performance prediction module based on El-Naggar's 1D loss modeling framework. It evaluates the complete head-capacity ($H - Q$), shaft power, hydraulic efficiency, and cavitation curves across a discretized range of flow coefficients.

Key Governing Equations

Non-Dimensional Flow & Head Iteration Loop

The solver iteratively determines internal flow parameter ϕ and theoretical head coefficient CH_0 by balancing impeller performance against volute and passage head losses:

$$\phi = -\frac{1}{2}\epsilon \cot(\beta_2) + \frac{1}{2}\sqrt{(\epsilon \cot(\beta_2))^2 + \frac{2}{\sigma x}}$$
$$CH_0 = \sigma \cdot \left(1 + \frac{\epsilon \cot(\beta_2)}{\phi}\right)$$

where $x = \frac{1}{1 + h_{lv,ratio} + h_{l,eye,ratio}}$ is the head loss attenuation factor.

Volute Hydraulic Loss Model

$$h_{lv,ratio} = C_{fv}CV_{3p}^2 + C_{dv}CV_{3d}^2 + CV_{3r}^2 + C_{f,th}CV_4^2$$

Friction factor (f_v) depends on local volute Reynolds number (Re_v):

$$f_v = \frac{0.3086}{\left[\log_{10}\left(\frac{6.9}{Re_v} + \left(\frac{k_{dhv}}{3.7}\right)^{1.11}\right)\right]^2}$$

Parasitic Power Losses & Shaft Head

Total shaft head coefficient CH_{shaft} combines ideal head, blade friction ($C_{h,lf}$), disc friction ($C_{h,disc}$), inlet recirculation ($C_{h,circ,in}$), and volumetric leakage ($C_{h,leakage}$):

$$CH_{shaft} = CH_0 + C_{h,lf} + C_{h,circ,in} + C_{h,leakage} + C_{h,disc}$$

$$\eta_{hyd} = \frac{CH}{CH_{shaft}}$$

Net Positive Suction Head (NPSH) & Cavitation Assessment

$$v_{suction} = \frac{Q}{A_{suction}}, \quad A_{suction} = \frac{\pi}{4}(D_{eye}^2 - D_{hub}^2)$$

$$NPSHA = \frac{p_{suction,abs} - p_{vapor}}{\rho g} + \frac{v_{suction}^2}{2g}$$

Cavitation Risk Flag: $NPSHa \leq NPSHr$

Underlying Assumptions

Static Loss Coefficients

Loss parameters (C_{dl} , C_{dv}) and passage surface roughness values (k_{volute} , $k_{impeller}$) remain constant across off-design operating conditions.

Enclosed Disk Friction Models

Disc friction torque loss relies on standard Daily & Nece empirical turbulent flow regimes for smooth, enclosed rotating disks.

Single-Phase Fluid Dynamics

Loss correlations assume single-phase liquid flow throughout the operating range. Performance breakdown due to vapor phase formation during severe cavitation is flagged as a failure state rather than modeled as two-phase flow.

Run 1D Hydraulic Solver with CoolProp

```
function [OutPump] = HydSolver_1D(ParSim)

plt_option = 0;
eAx_option = 0;
g = 9.81;
```

Fluid Properties (CoolProp Integration)

```
fluidProps = getFluidProperties2(ParSim);
rho = fluidProps.rho;
Nu = fluidProps.nu;
```

Pump Geometry & Operating Parameters

```
d2 = ParSim.pump.d2;
N = ParSim.operating.N_rpm;
b2 = ParSim.pump.b2;
d1 = ParSim.pump.Imp_inDia;
b1 = ParSim.pump.b1;
Z_LA = ParSim.pump.Z_LA;
d3_i = ParSim.pump.d3_i;
b3_i = ParSim.pump.b3_i;
deye = ParSim.pump.deye;
alphaTH = ParSim.pump.alphaTH;
L_throat = ParSim.pump.L_throat;

Beta1_deg = ParSim.pump.Beta_1;
Beta2_deg = ParSim.pump.Beta_2;
```

El-Naggar correlations use transformed angles (180 - Beta)

```
Beta2 = pi - deg2rad(Beta2_deg);
```

```
Beta1 = pi - deg2rad(Beta1_deg);
```

Fixed / Calibration Constants

```
cdl = getConst2(ParSim, 'cdl', 0.60);
Cdv = getConst2(ParSim, 'ceye', 1.00);
k_volute_i = getConst2(ParSim, 'k_volute_i', 1.0e-4);
k_impeller_i = getConst2(ParSim, 'k_impeller_i', 1.0e-4);
NPoints = getConst2(ParSim, 'NPoints', 80);
slipLossMulti = getConst2(ParSim, 'slipLossMulti', 1.0);
Cd_impeller = getConst2(ParSim, 'Cd_impeller', 0.02);

y_gap = getConst2(ParSim, 'y_gap', 0.50);
yc_i = getConst2(ParSim, 'yc_i', 0.50);
e1 = getConst2(ParSim, 'e1', 1.5);

drotor = getConst2(ParSim, 'D_o_bi', d2);
di_stator = getConst2(ParSim, 'D_i_stator', 15.0);
Motor_D_o = getConst2(ParSim, 'D_o', d2);
Rotor_Di = getConst2(ParSim, 'Rotor_Di', d1);
Imp_inDia = d1;

if isfield(ParSim.operating, 'P_in')
    dp_req = ParSim.operating.P_in;
elseif isfield(ParSim.operating, 'dp_target_bar')
    dp_req = ParSim.operating.dp_target_bar;
elseif isfield(ParSim.operating, 'dp_target_Pa')
    dp_req = ParSim.operating.dp_target_Pa / 1e5;
else
    dp_req = getConst2(ParSim, 'P_in_default_bar', 1.0);
end

alphavolute = abs((atan(((d1/d2)/(pi*(d3_i/d2)))) - (tan(deg2rad(alphaTH))/pi)) -
deg2rad(0.05)) * 180 / pi);
```

Slip Loss - Eddy Circulation Model

```
e_limit = exp(-8.16 * sin(Beta2) / Z_LA);
if e_limit >= (d1 / d2)
    sigma_0 = 1 - (sqrt(sin(Beta2)) / (Z_LA^0.7));
else
    sigma_0 = 1 - (sqrt(sin(Beta2)) * (1 - (((d1/d2) - e_limit) / (1 -
e_limit)))^0.3333) / (Beta1 * (Z_LA^0.7)));
end
```

Volute Parameters

```
alpha_v = alphavolute;
yc = yc_i;
Cd1 = cdl;
```

```

d_eye = deye;
d_th = d_eye;
Theta_throat = rad2deg(2 * atan((d_th - b3_i) * 0.5 / L_throat));
k_volute = k_volute_i;
k_impeller = k_impeller_i;
Nu_propellant = Nu;

u2 = pi * N * d2 * 0.001 / 60;      % Tip velocity (m/s)
Re2 = u2 * d2 * 0.001 * 0.5 / Nu_propellant;
dhv_d2 = 1 / ((0.5 / ((b2/b2)*(b2/d2))) + (1 / (8 * pi * (d3_i/d2) *
sin(deg2rad(alpha_v)) / Z_LA)));
k_dhv = (k_volute / d2) / dhv_d2;
Lv_D2 = 0.5 * pi * d3_i / (d2 * cos(deg2rad(alpha_v)));
Cf_th = 0.5 + (2.6 * sin(deg2rad(0.5 * Theta_throat)));
epsilon2 = 1 - (Z_LA * e1 / (pi * d2));

```

Iteration Loop Setup

```

eta_v_ass = 1;
sigma = sigma_0;
sigma_iteration = sigma_0;
eta_vol_iteration = eta_v_ass;
x = 1;
x_iteration = sigma_0;

eta_vector = zeros(1, NPoints);
x_vector = zeros(1, NPoints);
sigma_vector = zeros(1, NPoints);
phi_vector = zeros(1, NPoints);

epsi_lower_limit = 0.002;
epsi_vector = linspace(epsi_lower_limit, 0.30, NPoints);

for j = 1:NPoints
    epsi = epsi_vector(j);
    for i = 1:NPoints
        phi = (-0.5 * epsi * (1/tan(Beta2))) + (0.5 * sqrt(((epsi * (1/
tan(Beta2)))^2) + (2 / (sigma * x))));
        y = 1 + (epsi * (1/tan(Beta2)) / phi);

        sigma = max(1 - ((1 - sigma_0) / y), 0.01) * slipLossMulti;
        CH_0 = sigma * y;

        CV_5 = 4 * epsilon2 * eta_v_ass * epsi / ((d3_i/d2) * (b3_i/b2) *
tan(deg2rad(alpha_v)));
        CV_4 = epsilon2 * eta_v_ass * epsi / ((d3_i/d2) * (b3_i/b2) *
tan(deg2rad(alpha_v)));
        CV_eye = CV_5 / eta_v_ass;
        CV_3p = CV_4 / cos(deg2rad(alpha_v));
        CV_3d = sigma * (phi + (epsi * cot(Beta2))) - CV_4;
    end
end

```

```

CV_3r = epsilon2 * eta_v_ass * epsi / ((d3_i/d2) * (b3_i/b2));

tmp_sqrt = (CH_0 - ((CV_4^2 - CV_5^2 + CV_eye^2) / (2 * phi^2)) - ((1 -
(d1^2/d2^2)*(d_eye^2/d2^2)) / 8));
tmp_sqrt = max(tmp_sqrt, 0.0001);
eta_vol_calc = max(1 - ((yc/d_eye) * ((d_eye/d1)^2) * ((d1/d2)^2) * sqrt(2)
* Cd1 * phi * sqrt(tmp_sqrt) / (b2 * epsilon2 * epsi / d2)), 0.005);

eta_v_ass = eta_vol_calc;
eta_vol_iteration = [eta_vol_iteration, eta_v_ass]; %#ok<AGROW>
sigma_iteration = [sigma_iteration, sigma]; %#ok<AGROW>

Re_v = 2 * CV_3p * dhv_d2 * Re2 / phi;
fv = 0.3086 / ((log10((6.9 / max(Re_v, eps)) + (((k_dhv)/3.7)^1.11)))^2);
C_fv = fv * (Lv_D2) / dhv_d2;
h_lv_ratio = (C_fv * (CV_3p^2)) + (Cdv * (CV_3d^2)) + (CV_3r^2) + (Cf_th *
(CV_4^2));
hl_eye_ratio = CV_eye^2;
x = 1 / (1 + h_lv_ratio + hl_eye_ratio);
x_iteration = [x_iteration, x]; %#ok<AGROW>
end
eta_vector(j) = eta_vol_calc;
x_vector(j) = x;
sigma_vector(j) = sigma;
phi_vector(j) = phi;
end

if plt_option == 1
figure; plot(sigma_iteration, 'bs'); title('Sigma Iteration');
figure; plot(eta_vol_iteration, 'r*'); title('Volumetric Efficiency Iteration');
figure; plot(x_iteration, 'mo'); title('X Iteration');
end

```

Head, Flow, and Pressure Curves

```

CH = x_vector .* sigma_vector .* (1 + (epsi_vector .* cot(Beta2) ./ phi_vector));
CH_inf = 1 + (epsi_vector .* cot(Beta2) ./ phi_vector);
H_inf = CH_inf .* (u2^2) / g;
H_actual = CH .* (u2^2) / g;
CQ = eta_vector .* epsilon2 * (pi^2) * (b2/d2) .* (epsi_vector ./ phi_vector);
Q_actual = CQ .* N * ((d2 * 0.001)^3) / 60;
Q_actual_lpm = Q_actual * 1000 * 60;
dP_actual = H_actual * rho * g / 1e5;

%% Power Losses Calculations
d2_small = d2 * epsilon2 * pi / Z_LA;
epsilon1 = 1 - (Z_LA * e1 / (pi * d1));
d1_small = d1 * epsilon1 * pi / Z_LA;

```

```

Wav = 2 * u2 * epsilon2 * (pi/Z_LA) * (b2/d2) .* epsi_vector ./ (phi_vector .*
((d1_small * b1 * b2 / (d2*b2*d2)) + (d2_small * b2 / (d2^2))));
D_hydraulic = 2 * ((d1_small * b1) + (d2_small * b2)) / (d1_small + b1 + d2_small +
b2);
Re_impeller = 2 * Wav .* D_hydraulic * Re2 / (u2 * d2_small);
k_dhi = k_impeller / D_hydraulic;
f_impeller = 0.3086 ./ ((log10((6.9 ./ max(Re_impeller, eps)) + (((k_dhi)/
3.7)^1.11))) .^ 2);
L_blade = 0.5 * d2 * (1 - (d1/d2)) / sin(0.5 * (Beta1 + Beta2));
C_h_lf = 4 * Cd_impeller * (L_blade / d2) * 0.5 .* ((Wav ./ u2) ./ (D_hydraulic /
d2));

% Disc Friction Losses
G = y_gap / (0.5 * d2);
if Re2 < 4.5e4
    if G > 1.62 * (Re2^(-5/11))
        F_d_friction = 18.5 * (G^0.1) * (Re2^-0.5) / pi;
    else
        F_d_friction = 10 / (G * Re2);
    end
elseif Re2 >= 4.5e4 && Re2 < 1.6e5
    if G < 188 * (Re2^-0.9)
        F_d_friction = 10 / (G * Re2);
    else
        F_d_friction = 18.5 * (G^0.1) * (Re2^-0.5) / pi;
    end
else
    if G < 188 * (Re2^-0.9)
        F_d_friction = 10 / (G * Re2);
    elseif G > 0.402 * (Re2^(-3/16))
        F_d_friction = 0.51 * (G^0.1) * (Re2^-0.2) / pi;
    else
        F_d_friction = 0.4 * (G^(-1/6)) * (Re2^-0.25) / pi;
    end
end

K_d_disc = F_d_friction / (40 * epsilon2 * b2 / d2);
if eAx_option == 1
    rotor_loss_fraction = ((0.5 * drotor)^5 - (0.5 * di_stator)^5) / ((0.5 * d2)^5);
    C_h_disc = (1 + rotor_loss_fraction) * K_d_disc .* phi_vector ./ (eta_vector .*
epsi_vector);
else
    C_h_disc = K_d_disc .* phi_vector ./ (eta_vector .* epsi_vector);
end

```

Inlet Shock Circulation Loss

```

C_h_circ_in = max(((1 - (epsilon1 * pi * sin(Beta1) / Z_LA)) * ((d1/d2)^2)) - ...
(cot(pi - Beta1) .* epsi_vector ./ (b1 * epsilon1 .* phi_vector ./ (b2 *
epsilon2))), 0);

```

Shaft Power and Hydraulic Efficiency

```
CH_0_vector = sigma_vector .* (1 + (epsi_vector .* cot(Beta2) ./ phi_vector));
C_h_leakage = ((1 ./ eta_vector) - 1) .* (CH_0_vector + C_h_lf + C_h_circ_in);
CH_shaft = CH_0_vector + C_h_lf + C_h_circ_in + C_h_leakage + C_h_disc;
Eta_Hydraulic_vector = CH ./ CH_shaft;
```

Operating Point Interpolation

```
Q_vec = Q_actual_lpm;
P_actual = dP_actual;
T_vector = P_actual .* Q_vec * 100 ./ (2 * pi * N .* Eta_Hydraulic_vector);

Imp_Pratio = 0.92;
Q_m3s = ParSim.operating.Q_in * 1e-3 / 60;
P_m = dp_req * 1e5 / (g * rho);
Nq = N * (Q_m3s^0.5) / (P_m^0.75);

Outer_dia = max(drotor, Motor_D_o);

% =====
% UNBREAKABLE INTERPOLATION BLOCK
% =====
try
    % 1. Strip any complex/imaginary math artifacts
    Q_real = real(Q_vec);
    Eta_real = real(Eta_Hydraulic_vector);

    % 2. Filter out NaN and Infinity
    valid = isfinite(Q_real) & isfinite(Eta_real);
    Q_valid = Q_real(valid);
    Eta_valid = Eta_real(valid);

    % 3. Filter out duplicate coordinates (which crash interp1)
    [Q_clean, unique_idx] = unique(Q_valid);
    Eta_clean = Eta_valid(unique_idx);

    % 4. Interpolate securely
    if length(Q_clean) >= 2
        pump_eff = interp1(Q_clean, Eta_clean, ParSim.operating.Q_in, 'linear',
'extrap');
    else
        pump_eff = 0.01; % Give GA a 1% efficiency penalty
    end
catch
    % If absolutely anything else fails mathematically, do not crash.
    pump_eff = 0.01;
end
% =====
```

```

pump_trq = interp1(Q_vec, T_vector, ParSim.operating.Q_in, 'linear', 'extrap');
pump_pressure = interp1(Q_vec, P_actual, ParSim.operating.Q_in, 'linear', 'extrap');

Fax = 0.1 * pump_pressure * ((pi*(Outer_dia/2)^2 - pi*(Rotor_Di/2)^2) - Imp_Pratio
* (pi*(Outer_dia/2)^2 - pi*(Imp_inDia/2)^2));

OutPump.efficiency = pump_eff;
OutPump.torque = pump_trq;
OutPump.pressure = pump_pressure;
OutPump.D_impeller = d2;
OutPump.F_axial = Fax;
OutPump.Nq = Nq;
OutPump.Q_vec = Q_vec;
OutPump.P_vec = P_actual;
OutPump.Torque_vec = T_vector;
OutPump.Eff_vec = Eta_Hydraulic_vector;

```

Cavitation / NPSH Check

```

p_suction_abs = ParSim.operating.p_suction_abs;
p_vapor = fluidProps.p_vapor;
NPSH_required = ParSim.operating.NPSH_required;

D_eye_m = deye * 0.001;
D_hub_m = getConst2(ParSim, 'D_hub', 0) * 0.001;
A_suction = max((pi/4) * (D_eye_m^2 - D_hub_m^2), eps);
v_suction = Q_m3s / A_suction;

NPSH_available = (p_suction_abs - p_vapor) / (rho * g) + (v_suction^2 / (2 * g));
NPSH_margin = NPSH_available - NPSH_required;
NPSH_ratio = NPSH_available / max(NPSH_required, eps);

OutPump.cavitation.p_suction_abs_Pa = p_suction_abs;
OutPump.cavitation.p_vapor_Pa = p_vapor;
OutPump.cavitation.v_suction_mps = v_suction;
OutPump.cavitation.NPSH_available_m = NPSH_available;
OutPump.cavitation.NPSH_required_m = NPSH_required;
OutPump.cavitation.NPSH_margin_m = NPSH_margin;
OutPump.cavitation.NPSH_ratio = NPSH_ratio;
OutPump.cavitation.cavitation_risk = NPSH_available <= NPSH_required;

if OutPump.cavitation.cavitation_risk
    OutPump.cavitation.status = 'FAIL: NPSH_available <= NPSH_required; Cavitation
risk predicted.';
else
    OutPump.cavitation.status = 'PASS: NPSH_available > NPSH_required.';
end

OutPump.calculated_angles.Beta1_deg = ParSim.pump.Beta_1;

```

```
OutPump.calculated_angles.Beta2_deg = ParSim.pump.Beta_2;
```

```
end
```

Helper Functions

```
function props = getFluidProperties2(ParSim)
    if isfield(ParSim, 'fluid') && isfield(ParSim.fluid, 'name') &&
~strcmp(ParSim.fluid.name, 'custom')
        try
            fluid = ParSim.fluid.name;
            T_K = ParSim.fluid.temperature_K;
            P_Pa = ParSim.fluid.pressure_Pa;

            props.rho = double(py.CoolProp.CoolProp.PropsSI('D', 'T', T_K, 'P',
P_Pa, fluid));
            mu = double(py.CoolProp.CoolProp.PropsSI('V', 'T', T_K, 'P', P_Pa,
fluid));
            props.nu = mu / props.rho;
            props.p_vapor = double(py.CoolProp.CoolProp.PropsSI('P', 'T', T_K, 'Q',
0, fluid));
            return;
        catch
            % Fallback
        end
    end

    props.rho = getConst2(ParSim, 'rho_propellant', 1200.0);
    props.nu = getConst2(ParSim, 'Nu', 5.0e-7);
    props.p_vapor = getConst2(ParSim, 'p_vapor_default', 1.0e5);
end

function value = getConst2(ParSim, name, defaultValue)
    if isfield(ParSim, 'constants') && isfield(ParSim.constants, name)
        value = ParSim.constants.(name);
    else
        value = defaultValue;
    end
end
```